

Unit-2

Regular Languages & Context Free Grammars

Regular Expression

- The language accepted by finite automata can be easily described by simple expressions called Regular Expressions.
- It is the most effective way to represent any language.
- The languages accepted by some regular expression are referred to as Regular languages.
- A regular expression can also be described as a sequence of pattern that defines a string.
- Regular expressions are used to match character combinations in strings.
- String searching algorithm used this pattern to find the operations on a string.

For instance:

- In a regular expression, a^* means zero or more occurrence of a. It can generate {e, a, aa, aaa, aaaa,}
- In a regular expression, x^+ means one or more occurrence of x. It can generate {x, xx, xxx, xxxx,}

□ REs: Formal Definition

We construct REs from primitive constituents (basic elements) by repeatedly applying certain recursive rules as given below. (In the definition)

Definition : Let S be an alphabet. The regular expressions are defined recursively as follows.

□ Basis :

- i) ϕ is a RE
- ii) ϵ is a RE
- iii) $\forall a \in S, a$ is RE.

These are called primitive regular expression i.e. Primitive Constituents

□ Recursive Step :

If r_1 and r_2 are REs over, then so are

- i) $r_1 + r_2$
- ii) $r_1 r_2$
- iii) r_1^*
- iv) (r_1)

□ **Closure :** r is RE over only if it can be obtained from the basis elements (Primitive REs) by a finite no of applications of the recursive step (given in 2).

Language described by REs : Each describes a language (or a language is associated with every RE). We will see later that REs are used to attribute regular languages.

Notation : If r is a RE over some alphabet then $L(r)$ is the language associate with r . We can define the language $L(r)$ associated with (or described by) a REs as follows.

1. ϕ is the RE describing the empty language i.e. $L(\phi) = \phi$.
2. ϵ is a RE describing the language $\{\epsilon\}$ i.e. $L(\epsilon) = \{\epsilon\}$.
3. $\forall a \in S$, a is a RE denoting the language $\{a\}$ i.e . $L(a) = \{a\}$.
4. If r_1 and r_2 are REs denoting language $L(r_1)$ and $L(r_2)$ respectively, then
 - i) $r_1 + r_2$ is a regular expression denoting the language $L(r_1 + r_2) = L(r_1) \cup L(r_2)$
 - ii) $r_1 r_2$ is a regular expression denoting the language $L(r_1 r_2) = L(r_1) L(r_2)$

iii) r_1^* is a regular expression denoting the language $L(r_1^*) = (L(r_1))^*$

iv) (r_1) is a regular expression denoting the language $L((r_1)) = L(r_1)$

Example : Consider the RE $0^*(0+1)$. Thus the language denoted by the RE is

$$L(0^*(0+1)) = L(0^*) L(0+1) \dots\dots\dots \text{by 4(ii)}$$

$$= L(0)^* L(0) \cup L(1)$$

By 4(i)

$$= \{\epsilon, 0, 00, 000, \dots\dots\dots\} \{0\} \cup \{1\}$$

By 3

$$= \{\epsilon, 0, 00, 000, \dots\dots\dots\} \{0, 1\}$$

By 4(i)

$$= \{0, 00, 000, 0000, \dots\dots\dots, 1, 01, 001, 0001, \dots\dots\dots\}$$

By 4(ii)

Examples 2.1

1. Write the regular expression for the language accepting all combinations of a's, over the set $\Sigma = \{a\}$
2. Write the regular expression for the language accepting all combinations of a's except the null string, over the set $\Sigma = \{a\}$
3. Write the regular expression for the language accepting all the string containing any number of a's and b's.
4. Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over $\Sigma = \{0, 1\}$.
5. Write the regular expression for the language starting and ending with a and having any combination of b's in between.
6. Write the regular expression for the language starting with a but not having consecutive b's.
7. Write the regular expression for the language accepting all the string in which any number of a's is followed by any number of b's is followed by any number of c's.
8. Write the regular expression for the language over $\Sigma = \{0\}$ having even length of the string.
9. Write the regular expression for the language having a string which should have atleast one 0 and atleast one 1.
10. Write the regular expression for the language containing the string in which every 0 is immediately followed by 11.
11. Write the regular expression for the language containing the string over $\{0, 1\}$ in which there are at least two occurrences of 1's between any two occurrences of 0's.
12. Write the regular expression for the language L over $\Sigma = \{0, 1\}$ such that all the string do not contain the substring 01.

Solutions to the examples of 2.1

1. $\{e, a, aa, aaa, \dots\}a^*$
2. $\{a, aa, aaa, \dots\}a^+$
3. $\{e, a, b, aaa, bb, aba, aab, bab, \dots\} (a+b)^*$
4. $\{10, 1110, 1010, 100, 1000\}1(0+1)^*0$
5. $\{aa, aba, abba, \dots\}ab^*a$
6. $\{a, aa, abab, aab, aba, \dots\}(a+ab)^*$
7. $a^*b^*c^*$
8. $\{e, 00, 0000, \dots\}(00)^*$
9. $[(0+1)^*0(0+1)^*1(0+1)^*] + [(0+1)^*1(0+1)^*0(0+1)^*]$
10. $\{011, 011011, 1, 11, 111, \dots\} (011+1)^*$
11. $((0111^*0)^*+1)^*$
12. $\{e, 1, 0, 10, 00, 11, 100, \dots\} 1^*0^*$

Precedence Rule

Consider the RE $ab + c$. The language described by the RE can be thought of either $L(a)L(b+c)$ or $L(ab) \cup L(c)$ as provided by the rules (of languages described by REs) given already. But these two represents two different languages leading to ambiguity. To remove this ambiguity we can either

- 1) Use fully parenthesized expression- (cumbersome) or
- 2) Use a set of precedence rules to evaluate the options of REs in some order. Like other algebras mod in mathematics.

For REs, the order of precedence for the operators is as follows:

- i) The star operator precedes concatenation and concatenation precedes union (+) operator.
- ii) It is also important to note that concatenation & union (+) operators are associative and union operation is commutative.

$$(R1r2)r3=r1(r2r3)$$

$$r1r2!= r2r1$$

Using these precedence rule, we find that the RE $ab+c$ represents the language $L(ab) \cup L(c)$ i.e. it should be grouped as $((ab)+c)$.

We can, of course change the order of precedence by using parentheses. For example, the language represented by the RE $a(b+c)$ is $L(a)L(b+c)$.

Example : The RE ab^*+b is grouped as $((a(b^*))+b)$ which describes the language $L(a)(L(b))^* \cup L(b)$

Example : The RE $(ab)^*+b$ represents the language $(L(a)L(b))^* \cup L(b)$.

Example : It is easy to see that the RE $(0+1)^*(0+11)$ represents the language of all strings over $\{0,1\}$ which are either ended with 0 or 11.

Example : The regular expression $r = (00)^*(11)^*1$ denotes the set of all strings with an even number of 0's followed by an odd number of 1's i.e. $L(r) = \{0^{2n}1^{2m+1} \mid n \geq 0, m \geq 0\}$

Note : The notation r^+ is used to represent the RE rr^* . Similarly, r^2 represents the RE rr , r^3 denotes r^2r , and so on.

An arbitrary string over $\Sigma = \{0,1\}$ is denoted as $(0+1)^*$.

Exercise : Give a RE r over $\{0,1\}$ s.t. $L(r) = \{w \in \Sigma^* \mid w \text{ has at least one pair of consecutive 1's}\}$

Solution : Every string in $L(r)$ must contain `11` somewhere, but what comes before and what goes after is completely arbitrary. Considering these observations we can write the REs as $(0+1)^*11(0+1)^*$.

Example : Considering the above example it becomes clear that the RE $(0+1)^*11(0+1)^* + (0+1)^*00(0+1)^*$ represents the set of strings over $\{0,1\}$ that contain the substring `11` or `00`.

Example : Consider the RE $0^*10^*10^*$. It is not difficult to see that this RE describes the set of strings over $\{0,1\}$ that contain exactly two 1's. The presence of two 1's in the RE and any no. of 0's before, between and after the 1's ensure it.

Example : Consider the language of strings over $\{0,1\}$ containing two or more 1's.

Solution : There must be at least two 1's in the RE somewhere and what comes before, between, and after is completely arbitrary. Hence we can write the RE as $(0+1)^*1(0+1)^*1(0+1)^*$. But following two REs also represent the same language, each ensuring presence of least two 1's somewhere in the string

i) $0^*10^*1(0+1)^*$

ii) $(0+1)^*10^*10^*$

Example : Consider a RE r over $\{0,1\}$ such that

$$L(r) = \{w \in \{0,1\}^* \mid w \text{ has no pair of consecutive 1's}\}$$

Solution : Though it looks similar to ex, it is harder to construct to construct. We observe that, whenever a 1 occurs, it must be immediately followed by a 0. This substring may be preceded & followed by any no of 0's. So the final RE must be a repetition of strings of the form: $00...0100...00$ i.e. 0^*100^* . So it looks like the RE is $(0^*100^*)^*$. But in this case the strings ending in 1 or consisting of all 0's are not accounted for. Taking these observations into consideration, the final RE is $r = (0^*100^*)(1 + \epsilon) + 0^*(1 + \epsilon)$.

Identity rules

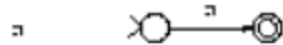
- $r + \Phi = \Phi + r = r$
- $r \in = \in r = r$
- $r\Phi = \Phi r = \Phi$
- $\in^* = \in$
- $r + r = r$
- $r^* r^* = r^*$
- $rr^* = r^* r$
- $rr^* = r^+$
- $(r^*)^* = r^*$
- $\in + rr^* = \in + r^* r = r^*$
- $(r + s)^* = (r^* s^*)^* = (r^* + s^*)^*$
- $(r + s)k = rk + sk$
- $(rs)^* r = r(sr)^*$

Converting a Regular Expression into a Deterministic Finite Automaton

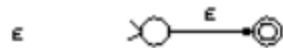
We will reduce the regular expression into smallest regular expressions and converting these to NFA and finally to DFA.

To convert a RE into a NFA there are only 5 rules, one for each type of RE:

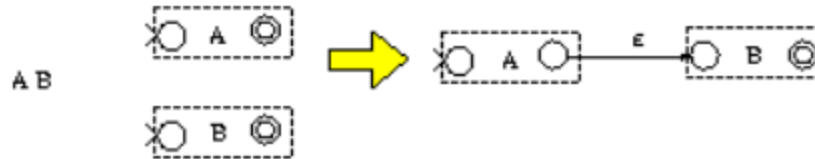
Case 1: RE is a



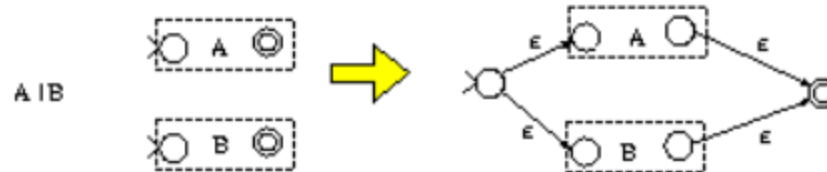
Case 2: RE is ϵ



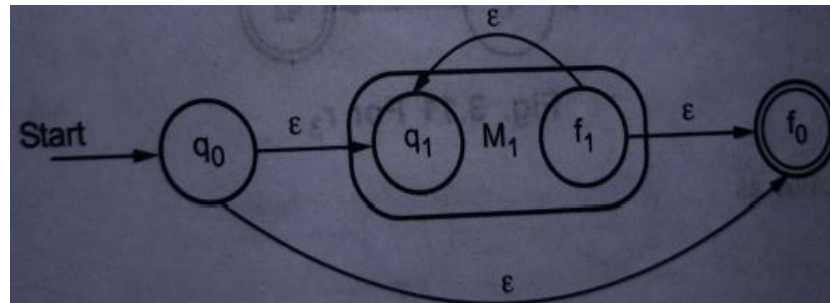
Case 3: RE is ab



Case 4: RE is $a+b$

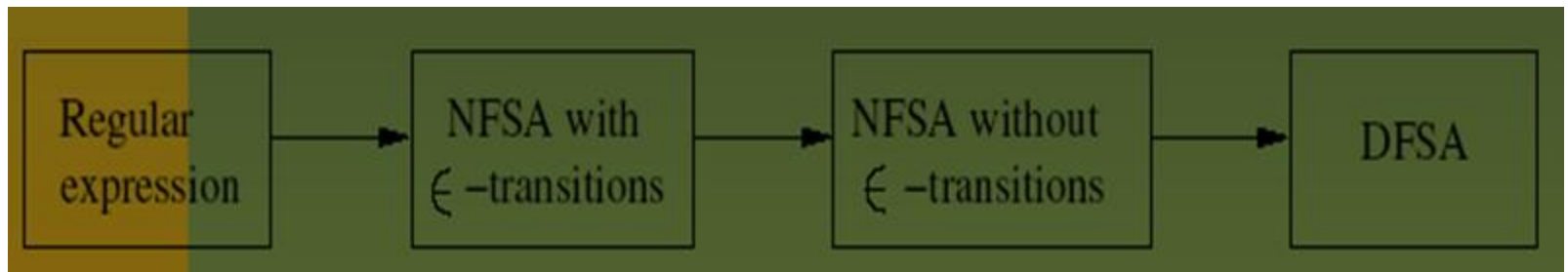


Case 5: RE is a^*



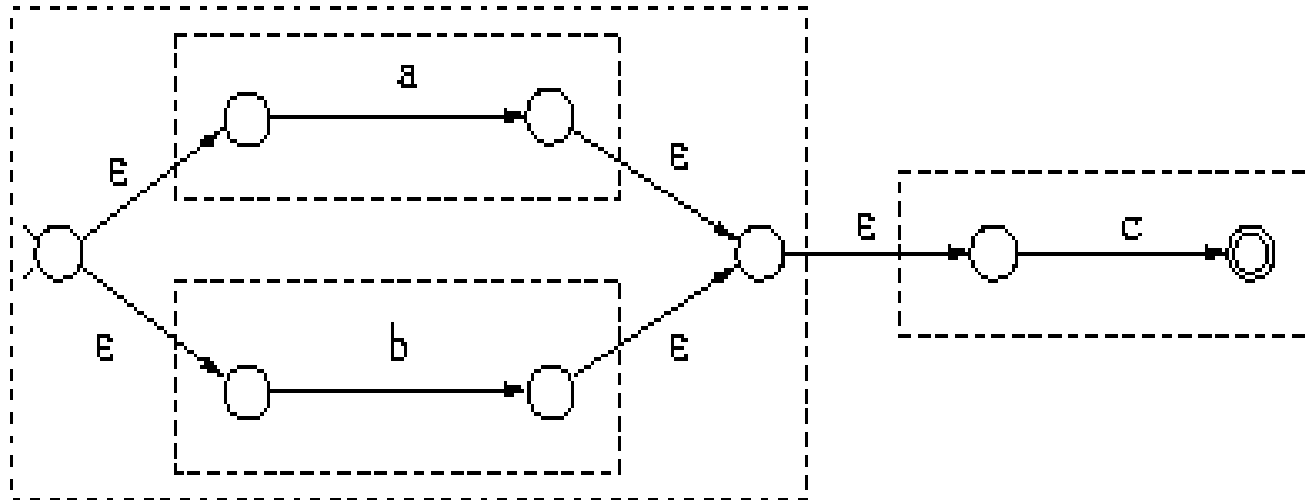
Method

- **Step 1** Construct an NFA with Null moves from the given regular expression.
- **Step 2** Remove Null transition from the NFA and convert it into its equivalent DFA.



Example 1: The RE $(a \mid b)c$ is mapped to the following NFA

- Construct NFA for each individual parts in the given RE
- We will concatenate three expressions "(a + b)" and "c".
- We obtain NFA with e-moves.
- Convert NFA with e-moves to NFA without e-moves.
- Convert NFA without e-moves to DFA.



Example 2: Convert the RE into its equivalent DFA – $1(0 + 1)^*0$

RE: $1(0 + 1)^*0$

- Construct NFA for each individual parts in the given RE
- We will concatenate three expressions "1", " $(0 + 1)^*$ " and "0".
- We obtain NFA with e-moves.
- Convert NFA with e-moves to NFA without e-moves.
- Convert NFA without e-moves to DFA.

3. Design a FA from given regular expression $10 + (0 + 11)0^*1$.

4. Design a NFA from given regular expression $1(1^*01^*01^*)^*$.

5. Construct the FA for regular expression $0^*1 + 10$.

6. Construct NFA for the regular expression $b + ba^*$

Arden's Theorem

In order to find out a regular expression of a Finite Automaton, we use Arden's Theorem along with the properties of regular expressions.

Statement of Arden's Theorem–

Let **P** and **Q** be two regular expressions. If **P** does not contain null string, then **R = Q + RP** has a unique solution that is **R = QP***

Proof –

$$\begin{aligned} R &= Q + (Q + RP)P \text{ [After putting the value } R = Q + RP\text{]} \\ &= Q + QP + RPP \end{aligned}$$

When we put the value of **R** recursively again and again, we get the following equation –

$$R = Q + QP + QP^2 + QP^3 \dots$$

$$R = Q (\epsilon + P + P^2 + P^3 + \dots)$$

$$R = QP^* \text{ [As } P^* \text{ represents } (\epsilon + P + P^2 + P^3 + \dots) \text{]}$$

Hence, proved.

Conversion of DFA to RE

Procedure to obtain Re for a given DFA

1. Let q_1 be the initial state.
2. There are $q_2, q_3, q_4 \dots q_n$ number of states. The final state may be some q_j where $j \leq n$.
3. Let α_{ji} represents the transition from q_j to q_i .
4. Calculate q_i such that

$$q_i = \alpha_{ji} * q_j$$

If q_j is a start state then we have:

$$q_i = \alpha_{ji} * q_j + \epsilon$$

5. Similarly, compute the final state which ultimately gives the regular expression 'r'.

Example 1: Construct a regular expression corresponding to the automata given below

Here the initial state and final state is q_1 .

The equations for the three states q_1 , q_2 , and q_3 are as follows –

$$q_1 = q_1a + q_3a + \epsilon \quad (\epsilon \text{ move is because } q_1 \text{ is the initial state})$$

$$q_2 = q_1b + q_2b + q_3b$$

$$q_3 = q_2a$$

Now, we will solve these three equations –

$$q_2 = q_1b + q_2b + q_3b$$

$$= q_1b + q_2b + (q_2a)b \quad (\text{Substituting value of } q_3)$$

$$= \underline{q_1b} + \underline{q_2(b + ab)}$$

$$R = \begin{matrix} Q & R & P \end{matrix}$$

$$R = QP^*$$

$$= q_1b(b + ab)^*$$

(Applying Arden's Theorem)

$$q_1 = q_1a + \underline{q_3a} + \epsilon$$

$$= q_1a + \underline{q_2aa} + \epsilon \quad (\text{Substituting value of } q_3)$$

$$= q_1a + \underline{q_1b(b + ab)^*aa} + \epsilon \quad (\text{Substituting value of } q_2)$$

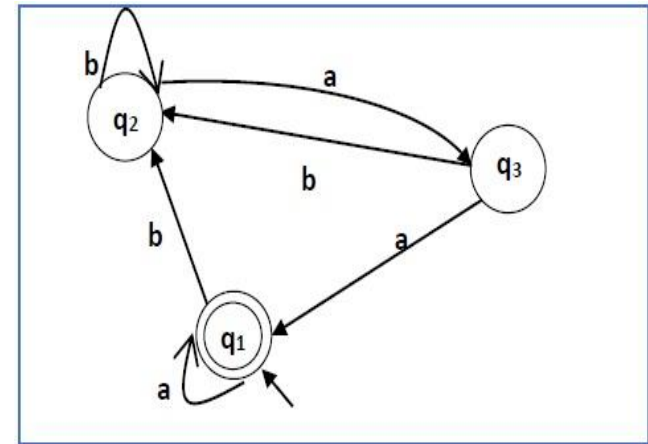
$$= \underline{q_1(a + b(b + ab)^*aa)} + \epsilon$$

$$\underline{R} = RP + Q$$

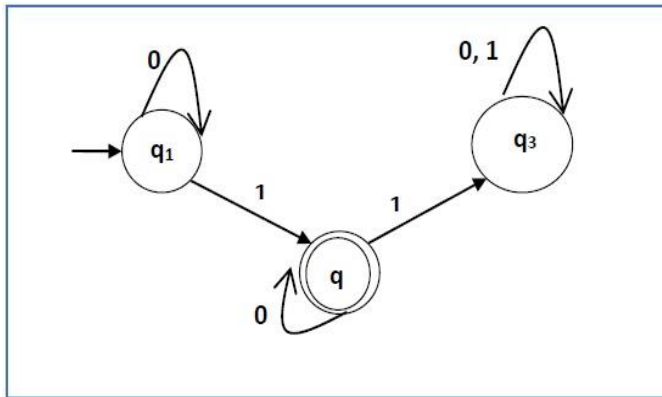
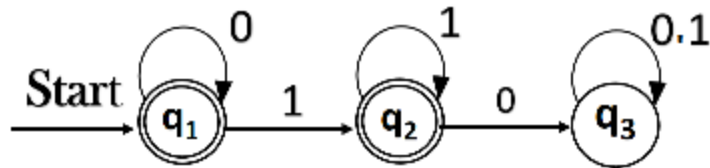
$$= \epsilon (a + b(b + ab)^*aa)^* \quad [\text{since } \epsilon r = r]$$

$$= \underline{(a + b(b + ab)^*aa)^*}$$

Hence, the regular expression is $(a + b(b + ab)^*aa)^*$.



Obtain the RE for the given DFA's



$$q_1 = q_1 0 + e$$

$$q_1 = e 0^*$$

$$q_2 = q_1 1 + q_2 1$$

$$Q_2 = q_1 1 1^*$$

Substitute q_1 in q_2

$$Q_2 = 0^* 1 1^*$$

$$R = r_1 + r_2$$

$$= 0^* + 0^* 1 1^*$$

$$R = 0^* + 0^* 1^+$$

$$q_3 = q_2 0 + q_3 0 + q_3 1$$

$$q_3 = q_2 0 + q_3 (0 + 1)$$

$$q_3 = q_2 0 (0 + 1)^*$$

Closure Properties of Regular Sets

Any set that represents the value of the Regular Expression is called a Regular Set. The language L is formed from them by operations then L is also regular. These properties are called closure properties.

Property 1. *The union of two regular set /language is regular.*

Property 2. *The intersection of two regular set is regular.*

Property 3. *The complement of a regular set is regular.*

Property 4. *The difference of two regular set is regular.*

Property 5. *The reversal of a regular set is regular.*

Property 6. *The closure of a regular set is regular.*

Property 7. *The concatenation of two regular sets is regular.*

Property 8. *A homomorphism of regular set is regular*

Property 9. *The inverse homomorphism of regular set is regular*

Context Free Grammar

Grammar

- Grammars denote syntactical rules for conversation in natural languages
- The theory of formal languages finds its applicability extensively in the fields of Computer Science.
- **Noam Chomsky** gave a mathematical model of grammar in 1956 which is effective for writing computer languages.

Grammar

A grammar **G** can be formally written as a 4-tuple (N, T, S, P) where –

- **N** or V_N is a set of variables or non-terminal symbols.
- **T** or Σ is a set of Terminal symbols.
- **S** is a special variable called the Start symbol, $S \in N$
- **P** is Production rules for Terminals and Non-terminals. A production rule has the form $\alpha \rightarrow \beta$, where α and β are strings on $V_N \cup \Sigma$ and least one symbol of α belongs to V_N .

Example

Grammar G1 – ($\{S, A, B\}$, $\{a, b\}$, S, $\{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}$)

Here, **S**, **A**, and **B** are Non-terminal symbols;

a and **b** are Terminal symbols

S is the Start symbol, $S \in N$

Productions, **P** : **S** $\rightarrow$ **AB**, **A** $\rightarrow$ **a**, **B** $\rightarrow$ **b**

Derivations from a Grammar

Strings may be derived from other strings using the productions in a grammar. If a grammar $\mathbf{G}$ has a production $\alpha \rightarrow \beta$, we can say that $\mathbf{x} \alpha \mathbf{y}$ derives $\mathbf{x} \beta \mathbf{y}$ in $\mathbf{G}$. This derivation is written as –

$$\mathbf{x} \alpha \mathbf{y} \Rightarrow_{\mathbf{G}} \mathbf{x} \beta \mathbf{y}$$

Example

Let us consider the grammar –

$G_2 = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$

Some of the strings (**aaabbb**) that can be derived are –

$S \Rightarrow \underline{a}Ab$	using production $S \rightarrow aAb$
$\Rightarrow aa\underline{A}bb$	using production $aA \rightarrow aAb$
$\Rightarrow aaa\underline{A}bbb$	using production $aA \rightarrow aAb$
$\Rightarrow aaabbb$	using production $A \rightarrow \epsilon$

The set of all strings that can be derived from a grammar is said to be the language generated from that grammar. A language generated by a grammar **G** is a subset formally defined by

$$L(G) = \{W \mid W \in \Sigma^*, S \Rightarrow_G W\}$$

If **L(G1) = L(G2)**, the Grammar **G1** is equivalent to the Grammar **G2**.

Example1

If there is a grammar

$$G: N = \{S, A, B\} \quad T = \{a, b\} \quad P = \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}$$

Here **S** produces **AB**, and we can replace **A** by **a**, and **B** by **b**. Here, the only accepted string is **ab**, i.e.,

$$L(G) = \{ab\}$$

Example2

Suppose we have the following grammar –

$$G: N = \{S, A, B\} \quad T = \{a, b\} \quad P = \{S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bB \mid b\}$$

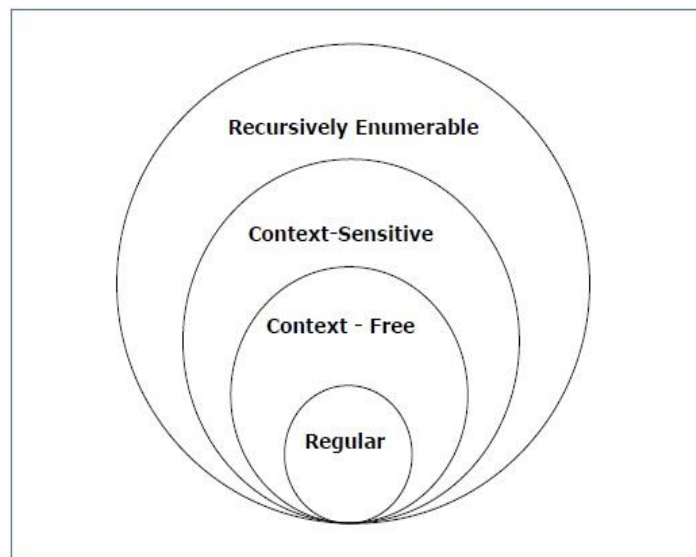
The language generated by this grammar –

$$\begin{aligned} L(G) &= \{ab, a^2b, ab^2, a^2b^2, \dots\} \\ &= \{a^m b^n \mid m \geq 1 \text{ and } n \geq 1\} \end{aligned}$$

Chomsky hierarchy of languages

According to Noam Chomsky, there are four types of grammars – Type 0, Type 1, Type 2, and Type 3. The following table shows how they differ from each other –

Grammar Type	Grammar Accepted	Language Accepted	Automaton
Type 0	Unrestricted grammar	Recursively enumerable language	Turing Machine
Type 1	Context-sensitive grammar	Context-sensitive language	Linear-bounded automaton
Type 2	Context-free grammar	Context-free language	Pushdown automaton
Type 3	Regular grammar	Regular language	Finite state automaton



The figure shows the scope of each type of grammar

Grammar Type	Production Rules	Language Accepted	Automata	Closed Under
Type-3 (Regular Grammar)	$A \rightarrow a$ or $A \rightarrow aB$ where $A, B \in N$ (no terminal) and $a \in T$ (Terminal)	Regular	Finite Automata	Union, Intersection, Complementation, Concatenation, Kleene Closure
Type-2 (Context Free Grammar)	$A \rightarrow \rho$ where $A \in N$ and $\rho \in (T \cup N)^*$	Context Free	Push Down Automata	Union, Concatenation, Kleene Closure
Type-1 (Context Sensitive Grammar)	$\alpha \rightarrow \beta$ where $\alpha, \beta \in (T \cup N)^*$ and $\text{len}(\alpha) \leq \text{len}(\beta)$ and α should contain at least 1 non terminal.	Context Sensitive	Linear Bound Automata	Union, Intersection, Complementation, Concatenation, Kleene Closure
Type-0 (Recursive Enumerable)	$\alpha \rightarrow \beta$ where $\alpha, \beta \in (T \cup N)^*$ and α contains at least 1 non-terminal	Recursive Enumerable	Turing Machine	Union, Intersection, Concatenation, Kleene Closure

Type - 3 Grammar

Type-3 grammars generate regular languages. Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal.

The productions must be in the form $X \rightarrow a$ or $X \rightarrow aY$ where $X, Y \in N$ (Non terminal) and $a \in T$ (Terminal)

The rule $S \rightarrow \epsilon$ is allowed if S does not appear on the right side of any rule.

Example

$X \rightarrow \epsilon ; X \rightarrow a \mid aY ; Y \rightarrow b$

Type - 2 Grammar

Type-2 grammars generate context-free languages.

The productions must be in the form $A \rightarrow \gamma$ where $A \in N$ (Non terminal) and $\gamma \in (T \cup N)^*$ (String of terminals and non-terminals).

These languages generated by these grammars are recognized by a non-deterministic pushdown automaton.

Example

$S \rightarrow Xa; X \rightarrow a; X \rightarrow aX; X \rightarrow abc; X \rightarrow \epsilon$

Type - 1 Grammar

Type-1 grammars generate context-sensitive languages. The productions must be in the form $\alpha A \beta \rightarrow \alpha \gamma \beta$

where $A \in N$ (Non-terminal) and $\alpha, \beta, \gamma \in (T \cup N)^*$ (Strings of terminals and non-terminals)

The strings α and β may be empty, but γ must be non-empty.

The rule $S \rightarrow \epsilon$ is allowed if S does not appear on the right side of any rule. The languages generated by these grammars are recognized by a linear bounded automaton.

Example

$AB \rightarrow AbBc$ $A \rightarrow bcA$ $B \rightarrow b$

Type - 0 Grammar

Type-0 grammars generate recursively enumerable languages. The productions have no restrictions. They are any phase structure grammar including all formal grammars. They generate the languages that are recognized by a Turing machine.

The productions can be in the form of $\alpha \rightarrow \beta$ where α is a string of terminals and non terminals with at least one non-terminal and α cannot be null. β is a string of terminals and non-terminals.

Example

$$S \rightarrow ACaB ; Bc \rightarrow acB ; CB \rightarrow DB ; aD \rightarrow Db$$

Context-Free Grammar(CFG)

Definition – A context-free grammar (CFG) consisting of a finite set of grammar rules is a quadruple **(N, T, P, S)** where

N is a set of non-terminal symbols.

T is a set of terminals where **$N \cap T = \text{NULL}$** .

P is a set of rules, **$P: N \rightarrow (N \cup T)^*$**

S is the start symbol.

Example

- The grammar **({A}, {a, b, c}, P, A)**, **$P: A \rightarrow aA, A \rightarrow abc$**
- The grammar **({S}, {a, b}, P, S)**, **$P: S \rightarrow aSa, S \rightarrow bSb, S \rightarrow \epsilon$**
- The grammar **({S, F}, {0, 1}, P, S)**, **$P: S \rightarrow 00S \mid 11F, F \rightarrow 00F \mid \epsilon$**

Example 1: Construct the CFG for the language having any number of a's over the set $\Sigma = \{a\}$.

the regular expression for the above language is

$$\text{r.e.} = a^*$$

The production rule for the Regular expression is as follows:

- $S \rightarrow aS$ rule 1
- $S \rightarrow \epsilon$ rule 2

Now if we want to derive a string "aaaaaa", we can start with start symbols.

- S
- aS
- aaS rule 1
- $aaaS$ rule 1
- $aaaaS$ rule 1
- $aaaaaS$ rule 1
- $aaaaaaS$ rule 1
- $aaaaaa\epsilon$ rule 2
- $aaaaaa$

The r.e. $= a^*$ can generate a set of string $\{\epsilon, a, aa, aaa, \dots\}$. We can have a null string because S is a start symbol and rule 2 gives $S \rightarrow \epsilon$.

Example: Construct the CFG for the language having any number of a's i.e., a^+ over the set $\Sigma = \{a\}$.

The production rule for the Regular expression is as follows:

- $S \rightarrow aS$ rule 1
- $S \rightarrow a$ rule 2

Example 2: Construct a CFG for the regular expression $(0+1)^*$.

The CFG can be given by,

Production rule (P):

- $S \rightarrow 0S \mid 1S$
- $S \rightarrow \epsilon$

The rules are in the combination of 0's and 1's with the start symbol. Since $(0+1)^*$ indicates $\{\epsilon, 0, 1, 01, 10, 00, 11, \dots\}$. In this set, ϵ is a string, so in the rule, we can set the rule $S \rightarrow \epsilon$.

Example 3: Construct a CFG for a language $L = \{wcw \mid \text{where } w \in (a, b)^*\}$.

The string that can be generated for a given language is $\{aaca, bcb, abcba, bacab, abbcbbba, \dots\}$

The grammar could be:

- $S \rightarrow aSa$ rule 1
- $S \rightarrow bSb$ rule 2
- $S \rightarrow c$ rule 3

Now if we want to derive a string "abbcbbba", we can start with start symbols.

- $S \rightarrow aSa$
- $S \rightarrow abSba$ from rule 2
- $S \rightarrow abbSbba$ from rule 2
- $S \rightarrow abbcbbba$ from rule 3

Derivation Tree

Generation of Derivation Tree

A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information a string derived from a context-free grammar.

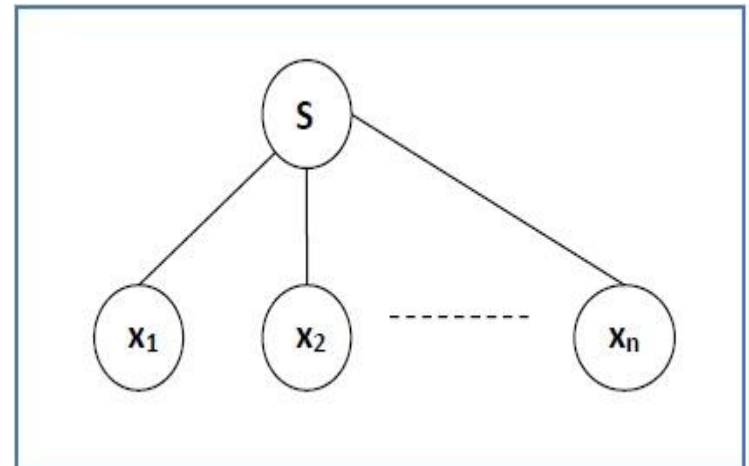
Representation Technique

Root vertex – Must be labeled by the start symbol.

Vertex – Labeled by a non-terminal symbol.

Leaves – Labeled by a terminal symbol or ϵ .

If $S \rightarrow x_1x_2 \dots x_n$ is a production rule in a CFG, then the parse tree / derivation tree will be as follows –



There are two different approaches to draw a derivation tree –

Top-down Approach –

- Starts with the starting symbol **S**

- Goes down to tree leaves using productions

Bottom-up Approach –

- Starts from tree leaves

- Proceeds upward to the root which is the starting symbol **S**

Derivation or Yield of a Tree

The derivation or the yield of a parse tree is the final string obtained by concatenating the labels of the leaves of the tree from left to right, ignoring the Nulls. However, if all the leaves are Null, derivation is Null.

Derivation Tree/Parse Tree

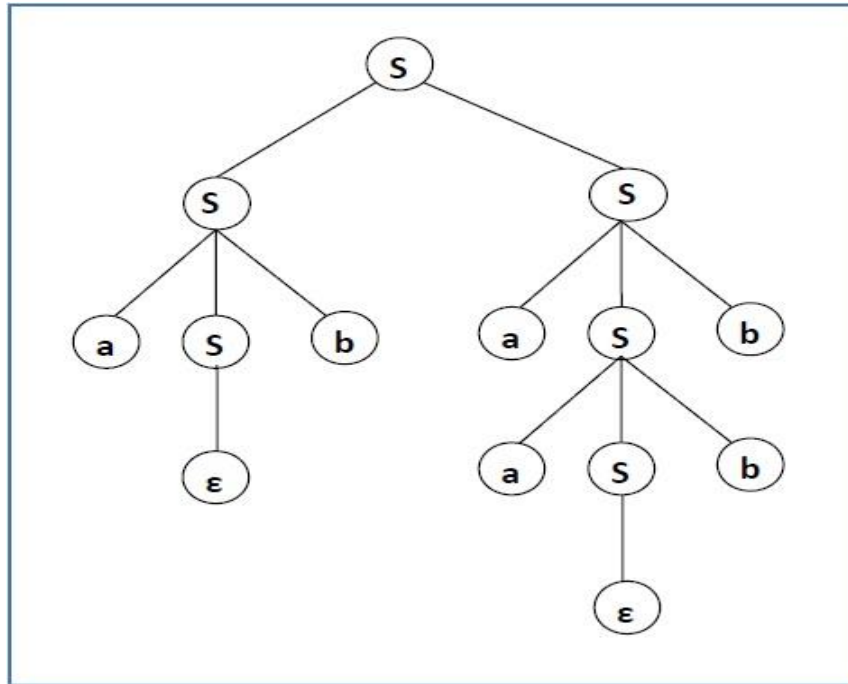
Example

Let a CFG $\{N, T, P, S\}$ be

$N = \{S\}$, $T = \{a, b\}$, Starting symbol = S , $P = S \rightarrow SS \mid aSb \mid \epsilon$

One derivation from the above CFG is “abaabb”

$S \Rightarrow \underline{S}S \Rightarrow a\underline{S}bS \Rightarrow ab\underline{S} \Rightarrow aba\underline{S}b \Rightarrow abaa\underline{S}bb \Rightarrow abaabb$



Sentential Form

Sentential Form and Partial Derivation Tree

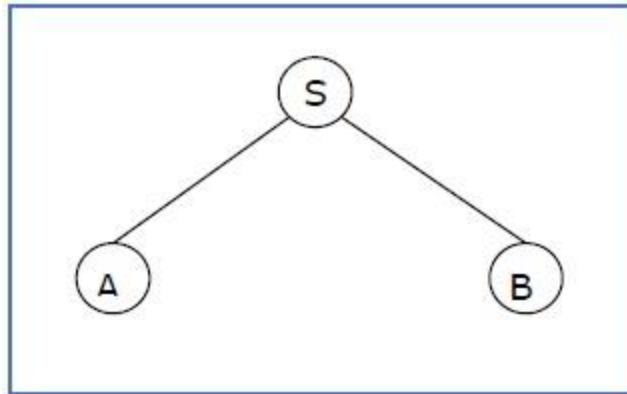
A partial derivation tree is a sub-tree of a derivation tree/parse tree such that either all of its children are in the sub-tree or none of them are in the sub-tree.

Example

If in any CFG the productions are –

$S \rightarrow AB$, $A \rightarrow aaA \mid \epsilon$, $B \rightarrow Bb \mid \epsilon$

the partial derivation tree can be the following –



If a partial derivation tree contains the root S , it is called a **sentential form**. The above sub-tree is also in sentential form.

Leftmost and Rightmost Derivation of a String

Leftmost derivation – A leftmost derivation is obtained by applying production to the leftmost variable in each step.

Rightmost derivation – A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Example

Let any set of production rules in a CFG be
 $X \rightarrow X+X \mid X^*X \mid X|a$;over an alphabet $\{a\}$.

The leftmost derivation for the string "**a+a*a**" may be –

$X \Rightarrow X+X \Rightarrow a+X \Rightarrow a + X^*X \Rightarrow a+a^*X \Rightarrow a+a^*a$

The rightmost derivation for the above string "**a+a*a**" may be –

$X \Rightarrow X^*X \Rightarrow X^*a \Rightarrow X+X^*a \Rightarrow X+a^*a \Rightarrow a+a^*a$

Example 1:

Production rules:

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow a \mid b$$

Input : $a - b + a$

The LMD for the input string is:

$$E \Rightarrow E + E$$

$$E \Rightarrow E - E + E$$

$$E \Rightarrow a - E + E$$

$$E \Rightarrow a - b + E$$

$$E \Rightarrow a - b + a$$

The RMD for the input string is :

$$E \Rightarrow E - E$$

$$E \Rightarrow E - E + E$$

$$E \Rightarrow E - E + a$$

$$E \Rightarrow E - b + a$$

$$E \Rightarrow a - b + a$$

Task is to draw the derivation tree for the above string.

Example: Derive the string "abb" for leftmost derivation and rightmost derivation using a CFG given by,

$$S \rightarrow AB \mid \varepsilon$$

$$A \rightarrow aB$$

$$B \rightarrow Sb$$

Leftmost derivation

S
 AB
 $\boxed{aB} B$
 $a \boxed{Sb} B$
 $\mathbf{a} \boxed{\varepsilon} bB$
 $ab \boxed{Sb}$
 $ab \boxed{\varepsilon} b$
 abb

Rightmost derivation

S
 AB
 $A \boxed{Sb}$
 $A \boxed{\varepsilon} b$
 $\boxed{aB} b$
 $a \boxed{Sb} b$
 $a \boxed{\varepsilon} bb$
 abb

Example:

Derive the string "aabbabba" for leftmost derivation and rightmost derivation using a CFG given by,

$$S \rightarrow aB \mid bA$$

$$S \rightarrow a \mid aS \mid bAA$$

$$S \rightarrow b \mid aS \mid aBB$$

Derive the string "00101" for leftmost derivation and rightmost derivation using a CFG given by,

$$S \rightarrow A1B$$

$$A \rightarrow 0A \mid \epsilon$$

$$B \rightarrow 0B \mid 1B \mid \epsilon$$

Given CFG obtain the derivation tree for the input string $a * b + c$

$$E = E + E$$

$$E = E * E$$

$$E = a \mid b \mid c$$

Draw a derivation tree for the string "bab" from the CFG given by

$$S \rightarrow bSb \mid a \mid b$$

Ambiguous Grammar

A grammar is said to be ambiguous if there exists more than one leftmost derivation or more than one rightmost derivation or more than one parse tree for the given input string.

If the grammar is not ambiguous, then it is called unambiguous.

Example: Let us consider a grammar G with the production rule

$E \rightarrow I$

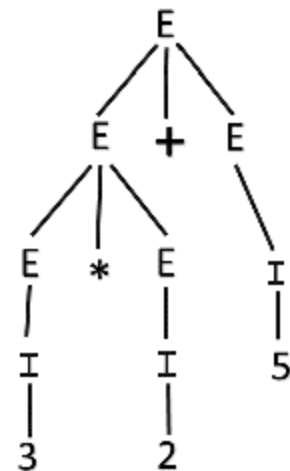
$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$I \rightarrow \epsilon \mid 0 \mid 1 \mid 2 \mid \dots \mid 9$

For the string "3 * 2 + 5", the above grammar can generate two parse trees by leftmost derivation



Ex1: Check whether the given grammar G is ambiguous or not.

$$E \rightarrow E + E$$
$$E \rightarrow E - E$$
$$E \rightarrow \text{id}$$

For the String "id + id - id"

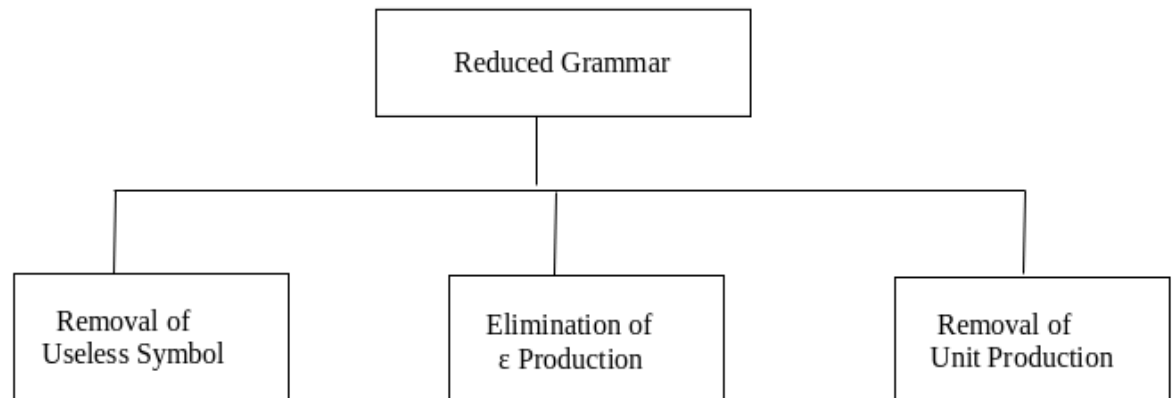
Ex2: Check whether the given grammar G is ambiguous or not.

$$S \rightarrow aSb \mid SS$$
$$S \rightarrow \varepsilon$$

For the string "aabb"

Minimization of CFG

- Having extra symbols, unnecessary increase the length of grammar.
- Simplification of grammar means reduction of grammar by removing useless symbols.
- The properties of reduced grammar are given below:
 - Each variable (i.e. non-terminal) and each terminal of G appears in the derivation of some word in L .
 - There should not be any production as $X \rightarrow Y$ where X and Y are non-terminal.
 - If ϵ is not in the language L then there need not to be the production $X \rightarrow \epsilon$.



Removal of Useless Symbols

- A symbol can be useless if it does not appear on the right-hand side of the production rule and does not take part in the derivation of any string.
 - Ex: $T \rightarrow aaB \mid abA \mid aaT$
 - $A \rightarrow aA$
 - $B \rightarrow ab \mid b$
 - $C \rightarrow ad$

Elimination of ϵ Production

The productions of type $S \rightarrow \epsilon$ are called ϵ productions. These type of productions can only be removed from those grammars that do not generate ϵ .

Example:

Remove the ϵ production from the following CFG by preserving the meaning of it.

$$S \rightarrow XYX$$

$$X \rightarrow 0X \mid \epsilon$$

$$Y \rightarrow 1Y \mid \epsilon$$

moving ϵ production, we are deleting the rule $X \rightarrow \epsilon$ and $Y \rightarrow \epsilon$. To preserve the meaning of CFG we are actually placing ϵ at the right-hand side whenever X and Y have appeared.

Let us take

$$S \rightarrow XYX$$

If the first X at right-hand side is ϵ . Then

$$S \rightarrow YX$$

Similarly if the last X in R.H.S. = ϵ . Then

$$S \rightarrow XY$$

If $Y = \varepsilon$ then

$S \rightarrow XX$

If Y and X are ε then,

$S \rightarrow X$

If both X are replaced by ε

$S \rightarrow Y$

Now,

$S \rightarrow XY \mid YX \mid XX \mid X \mid Y$

Now let us consider

$X \rightarrow 0X$

If we place ε at right-hand side for X then,

$X \rightarrow 0$

$X \rightarrow 0X \mid 0$

Similarly $Y \rightarrow 1Y \mid 1$

n rewrite the CFG with removed ε production as

$S \rightarrow XY \mid YX \mid XX \mid X \mid Y$

$X \rightarrow 0X \mid 0$

$Y \rightarrow 1Y \mid 1$

Removing Unit Productions

Unit productions are the productions in which one non-terminal gives another non-terminal. Use the following steps to remove unit production:

Step 1: To remove $X \rightarrow Y$, add production $X \rightarrow a$ to the grammar rule whenever $Y \rightarrow a$ occurs in the grammar.

Step 2: Now delete $X \rightarrow Y$ from the grammar.

Step 3: Repeat step 1 and step 2 until all unit productions are removed.

For example:

$$S \rightarrow 0A \mid 1B \mid C$$
$$A \rightarrow 0S \mid 00$$
$$B \rightarrow 1 \mid A$$
$$C \rightarrow 01$$

Solution:

$S \rightarrow C$ is a unit production. But while removing $S \rightarrow C$ we have to consider what C gives. So, we can add a rule to S .

$$S \rightarrow 0A \mid 1B \mid 01$$

Similarly, $B \rightarrow A$ is also a unit production so we can modify it as

$$B \rightarrow 1 \mid 0S \mid 00$$

Thus finally we can write CFG without unit production as

$$S \rightarrow 0A \mid 1B \mid 01$$
$$A \rightarrow 0S \mid 00$$
$$B \rightarrow 1 \mid 0S \mid 00$$

Normal forms

To obtain the grammar in a specific order.

Normalize the grammar.

There should be fixed no of terminals and non-terminals in CFG.

Two types of Normal forms.

- 1) Chomsky's Normal Form(CNF)
- 2) Greibach Normal Form(GNF)

Chomsky's Normal Form (CNF)

A CFG(context free grammar) is in CNF(Chomsky normal form) if all production rules satisfy one of the following conditions:

- Start symbol generating ϵ . For example, $A \rightarrow \epsilon$.
- A non-terminal generating two non-terminals. For example, $S \rightarrow AB$.
- A non-terminal generating a terminal. For example, $S \rightarrow a$.

Ex: { $S \rightarrow AB$, $S \rightarrow c$, $A \rightarrow a$, $B \rightarrow b$ }

Ex: { $S \rightarrow aA$, $A \rightarrow a$, $B \rightarrow c$ }

For example, production $S \rightarrow aA$ can be decomposed as:

$S \rightarrow RA$

$R \rightarrow a$

Eliminate RHS with more than two non-terminals.

For example, $S \rightarrow ASB$ can be decomposed as:

$S \rightarrow AR$

$R \rightarrow SB$

Ex:

$S \rightarrow aaaaS$

$S \rightarrow aaaa$

Examples

Convert the following grammars into CNF

1) $S \rightarrow aAD$

$A \rightarrow aB \mid bAB$

$B \rightarrow b$

$D \rightarrow d$

2) $S \rightarrow a \mid b \mid aSS$

3) $S \rightarrow ABA$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow bB \mid \varepsilon$

4) $S \rightarrow aAS \mid a$

$A \rightarrow SbA \mid SS \mid ba$

Greibach Normal Form (GNF)

A CFG(context free grammar) is in GNF(Greibach normal form) if all the production rules satisfy one of the following conditions:

- Start symbol generating ϵ . For example, $A \rightarrow \epsilon$.
- A non-terminal generating a terminal. For example, $A \rightarrow a$.
- A non-terminal generating a terminal which is followed by any number of non-terminals. For example, $S \rightarrow aASB$.

For example:

- $G1 = \{S \rightarrow aAB \mid aB, A \rightarrow aA \mid a, B \rightarrow bB \mid b\}$

To obtain the grammar in GNF, two Lemma's are used.

- Lemma 1 and Lemma 2:

Lemma 1:

Let $G = (V_N, \Sigma, P, S)$ be a CFG, let $A \rightarrow B^s$ be an A-production in P . Let the B-productions be $B \rightarrow \beta_1 / \beta_2 / \dots / \beta_s$. Define $P_1 = (P - \{A \rightarrow B^s\}) \cup \{A \rightarrow \beta_i / 1 \leq i \leq s\}$

- Lemma 2

If $A \rightarrow \alpha_1 / \alpha_2 / \dots / \beta_1 / \beta_2$ then $A \rightarrow \beta_1 / \beta_2 / \dots$

$A \rightarrow \beta_1 z / \beta_2 z / \dots / \beta_s z$
The set of z -productions in P_1 are
(ii) $z \rightarrow \alpha_1 / \alpha_2 / \dots / \alpha_n$
 $z \rightarrow \alpha_1 z / \alpha_2 z / \dots / \alpha_n z$

CONVERT the following CFG TO GNF

$$1) S \rightarrow ABA$$

$$A \rightarrow aA \mid \varepsilon$$

$$B \rightarrow bB \mid \varepsilon$$

Remove Useless, null productions and unit productions if any.

$$2) S \rightarrow SS$$

$$S \rightarrow 0S1 \mid 01$$

$$3) S \rightarrow CA$$

$$A \rightarrow a$$

$$C \rightarrow aB \mid b$$

$$4) S \rightarrow AA \mid 0$$

$$A \rightarrow SS \mid 1$$